2024 32nd International Conference on Enabling Technologies: Infrastructure for Collaborative Enterprises (WETICE) | 979-8-3315-0587-5/24/$31.00 ©2024 IEEE | DOI: 10.1109/WETICE64632.2024.00020

RVElastic: a Runtime Verification Framework for Microservice Systems

Stefano Murino
Lead Architect
Doxee SPA
Modena, Italy
smurino@doxee.com

Angelo Ferrando
Department of Physics, Informatics and Mathematics
University of Modena and Reggio Emilia
Modena, Italy
angelo.ferrando@unimore.it

Giacomo Cabri
Department of Physics, Informatics and Mathematics
University of Modena and Reggio Emilia
Modena, Italy
giacomo.cabri@unimore.it

***Abstract*—This paper proposes RVElastic, a Runtime Verification prototype framework for monitoring microservice systems. We present its general architecture and report a possible instantiation wherein Apache Kafka is utilised to create the system and its instrumentation, while OpenSearch is employed as a means to analyse and visualise the verification results. We experiment with RVElastic on a simple case study as a proof of concept and report the results in terms of the overhead introduced by the addition of monitors in the microservice system.**

***Index Terms*—Apache Kafka, OpenSearch, Runtime Verification**

I. Introduction

Microservice architecture has emerged as a prominent paradigm in software engineering [1], [2], offering numerous benefits such as scalability, flexibility, and rapid development. By breaking down applications into small, independently deployable services, microservices enable organisations to efficiently manage complex systems and adapt to changing business requirements. However, as the adoption of microservices continues to grow, so do the challenges associated with ensuring their reliability, robustness, and safety, particularly in safety-critical domains.

In safety-critical domains such as healthcare, transportation, and finance, the reliability and correctness of software systems are of paramount importance [3]. Any failure or malfunction in these systems can have serious consequences, including financial losses, compromised data integrity, and even threats to human life. Therefore, the deployment of microservices in such contexts necessitates rigorous verification and validation to guarantee their adherence to safety standards and regulatory requirements.

One of the key challenges in ensuring the safety of microservice-based systems lies in their distributed nature. Unlike monolithic architectures, where all components reside within a single executable, microservices operate as independent units that communicate with each other over a network. While this distributed nature offers advantages in terms of scalability and fault isolation, it also introduces complexities in terms of coordination, data consistency, and fault tolerance.

Moreover, the dynamic and heterogeneous nature of microservice environments further complicates the task of ensuring system safety. Microservices may be developed using different programming languages, frameworks, and technologies, making it challenging to maintain consistency and coherence across the entire system. Additionally, the deployment of microservices in cloud environments introduces additional layers of abstraction and potential points of failure.

To address these challenges and ensure the safety of microservice-based systems, rigorous verification techniques are essential. Runtime Verification (RV) [4] has emerged as a promising approach for monitoring and analyzing the behavior of distributed systems in real-time. By instrumenting the system with monitors that observe its execution and enforce specified properties, RV enables early detection of anomalies, violations, and potential safety hazards.

In this paper, we propose RVElastic, a Runtime Verification framework tailored for monitoring microservice systems. We present its architecture, which leverages Apache Kafka for system instrumentation and OpenSearch for analysis and visualization of verification results. Our approach aims to provide a scalable and flexible solution for ensuring the safety and reliability of microservice-based systems in safety-critical domains. We demonstrate the effectiveness of RVElastic through experimentation on a simple case study, evaluating the overhead introduced by the addition of monitors in the microservice system. Through this research, we aim to contribute to the advancement of techniques for verifying the safety of distributed software systems, particularly in safety-critical contexts.

II. Related Work

Recent research focuses on Runtime Verification of actor-based and distributed systems.

Yaseen et al. [5] introduce Aragog, a scalable runtime verification system for complex distributed network functions like firewalls and NATs. It efficiently detects violations of properties in running systems and demonstrates effectiveness in real-world network functions.

Shafiei et al. [6] present MESA, a runtime verification tool implemented in Scala DSL. MESA utilises concurrent monitors based on the actor model to check properties specified in

979-8-3315-0587-5/24/$31.00 ©2024 IEEE
DOI 10.1109/WETICE64632.2024.00020

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 14:42:31 UTC from IEEE Xplore. Restrictions apply.

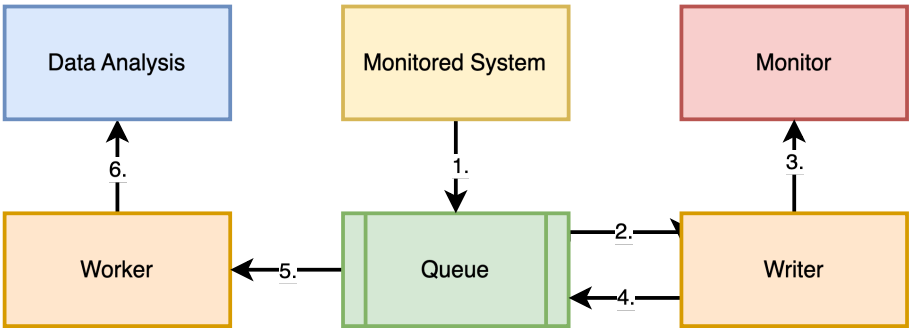


Fig. 1: RVElastic architecture.

data parameterised temporal logic and state machines, showing effectiveness in monitoring live US airspace data streams.

Kauffman et al. [7] propose a notation for efficiently processing telemetry data from spacecraft. Implemented in Scala as nfer, it employs parallel executing actors communicating via a publish-subscribe model, focusing on computing abstractions rather than checking execution traces.

Basin et al. [8] present a framework for scalable online monitoring of distributed systems. It extends existing techniques to handle multiple observation sequences in parallel, addressing challenges such as scalability and handling partially-ordered observations.

Dams et al. [9] explore enhancing dynamic documentation in Runtime Verification through data analysis techniques, providing examples such as state reconstruction and event visualisation.

III. Architecture

Figure 1 illustrates the overall architecture schema, while Figure 2 depicts a diagram of the architecture with the utilised technologies.

A. Data Generation

The first component of the architecture is the system under analysis (SUA), also referred to as the **Monitored System**. This is the system we are interested in analysing. Since we are focusing on microservice-based systems, the data we analyse consists of messages exchanged among the different components of the system. These messages are formatted in JSON and propagated to the monitor(s) for verification. The structure of these JSON events is entirely agnostic from the monitor implementation perspective; the verification system does not need to know how such messages have been produced but only needs to gather and analyse them as soon as they are available. To avoid impacting the performance of the SUA, data analysis is carried out using an asynchronous queue system. This ensures that the engagement is entirely asynchronous, and any anomalies or differences in component availability policies do not affect the work of the two systems.

Note that, this step is commonly referred in Runtime Verification as instrumentation [10], which is the act of modifying the SUA in order to gather all the data needed for monitoring the system.

B. Data Ingestion and Verification

The **Worker** component is responsible for reading the messages from the input queue, which is filled with events produced by the SUA. This component interacts with one (or multiple) monitor(s) via Websocket calls, with each call including the received message in the body. The **Monitor** component is external to the system and is designed to provide feedback on the formal correctness of the messages received by the worker component, giving a Boolean verdict. Upon receiving the monitor's reply, the worker component enriches the original message by adding a new field with the received Boolean verdict. The key of this value is configurable during the deployment of the component for better customisation in the analysis phase. The last step of the worker is to propagate the enriched message to another queue. This latter queue is read by a component named **Writer**, which is responsible for propagating the enriched message to a Data Analysis component without any modification of the data.

Note that, RVElastic is completely agnostic regarding the monitor component. The only requirement the monitor component needs to satisfy is its capability to interact with the worker component through Websocket. Apart from this, the actual verification at runtime can be achieved in any manner and does not affect the rest of the system. For this reason, RVElastic can be considered formalism-agnostic.

C. Data Analysis

The **Data Analysis** component is responsible for storing the data. Various types of analysis can be performed on the data using counters, graphs, and tables to display the results of the analysis. In our architecture, this component is extended to consider the monitor's outcomes. This enables further analysis of the system's behaviour by considering the satisfaction or violation of formal properties monitored at runtime.

IV. Implementation

To implement the architecture described in Section III, Java microservices based on the Spring Boot framework [11] were chosen. A repository containing a prototype implementation

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 14:42:31 UTC from IEEE Xplore. Restrictions apply.

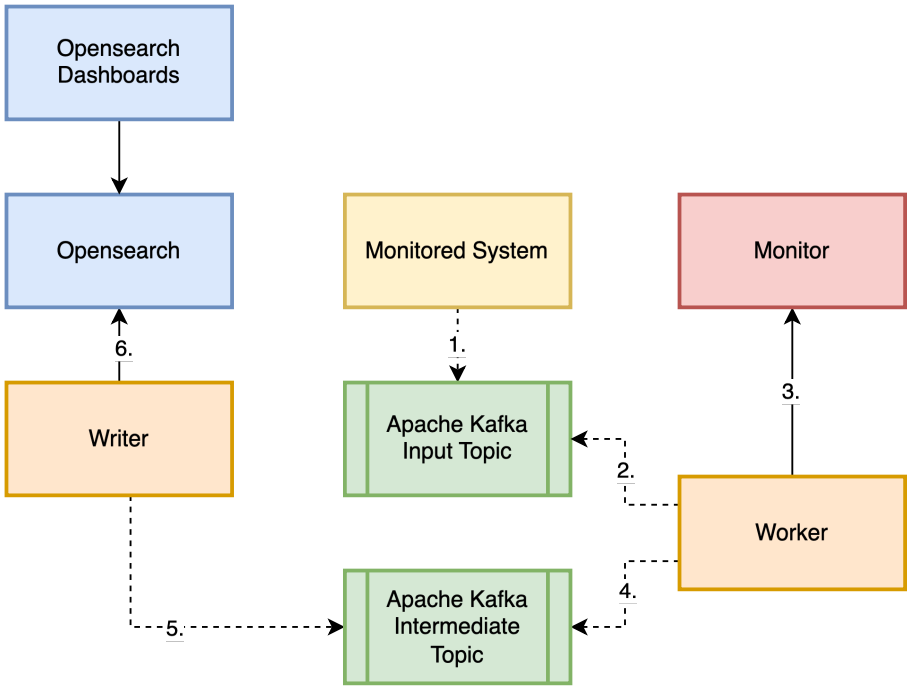


Fig. 2: A possible instantiation of the RVElastic architecture.

of RVElastic can be found at https://github.com/stefno83/rvelastic.

For the implementation of queues, the Apache Kafka [12] Pub/Sub system was selected. This choice allows events to be read not only by our system but also by other systems. Unlike traditional queues that handle messages sequentially and are consumed by a single recipient, the pub/sub system enables the distribution of messages from one or more publishers to one or more subscribers interested in specific topics without retaining messages sent before the subscriber subscribes. This allows multiple workers to be hooked into the system, analysing the same message but for different formal properties. At deployment time, through configuration it is possible to specify which JSON fields to use to store the monitor's outcome and the address where the external monitor can be found.

The use of Spring Boot and Kafka technologies helps build a more robust and error-resilient architecture. Spring Boot provides extensions that facilitate transaction and error management. Developers can easily define Kafka listeners and writers using annotations without needing detailed knowledge of Kafka's implementation. This approach is crucial in a modular system, enabling transparent software version updates and the ability to change the pub/sub engine without altering the code.

For Data Analysis, OpenSearch [13] with OpenSearch Dashboard has been chosen. OpenSearch is a standard technology for log analysis and creating reporting dashboards based on logs, making it suitable for our purpose. The creation of dashboards enables the display of the monitored system's behaviour concerning the analysed formal property using counters, tables, and graphs.

All artifacts are compiled into Docker Images, allowing the system to be installed on Docker [14], Kubernetes [15], or related systems. This enables integration with any customer's microservice-based system.

A. Technologies

All technologies used are open-source and are standard for use in this project, distributed under the 'Apache License 2.0'. The Apache License 2.0 is a permissive open-source software license that grants recipients extensive rights to use, modify, and distribute the software, as long as they comply with certain conditions. It allows for the software to be distributed under different terms or licenses if desired, and includes an express grant of patent rights from contributors to users.

1) OpenSearch and OpenSearch Dashboard: OpenSearch is an open-source search and analytics suite that offers a robust and scalable solution for searching, analysing, and visualising large volumes of data. It provides a flexible and customizable platform for building search applications, enabling users to index diverse data sources and perform advanced search operations efficiently. OpenSearch is designed to be highly extensible, allowing developers to integrate additional functionalities and plugins tailored to their specific needs. OpenSearch Dashboard is a user interface tool that accompanies OpenSearch, offering intuitive visualisations and dashboards for exploring and interpreting search results and analytics data. It provides a user-friendly interface for creating, managing, and sharing interactive dashboards, charts, and graphs based on indexed data. With OpenSearch Dashboard, users can easily customise

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 14:42:31 UTC from IEEE Xplore. Restrictions apply.

the layout and design of their dashboards, add various data visualisations, and drill down into specific data subsets for deeper analysis. Both OpenSearch and OpenSearch Dashboard are built on open standards and APIs, facilitating seamless integration with existing systems and applications. They support a wide range of data formats and sources, including structured and unstructured data, logs, metrics, and more. Additionally, OpenSearch and OpenSearch Dashboard are highly scalable and resilient, capable of handling large-scale deployments and delivering high performance even under heavy workloads. In summary, OpenSearch and OpenSearch Dashboard together provide a powerful, flexible, and user-friendly solution for building and managing search and analytics applications. With their open-source nature and extensive features, they empower developers and organisations to leverage the full potential of their data for insights, decision-making, and innovation.

2) Apache Kafka: Apache Kafka is a distributed event streaming platform that serves as a high-throughput, fault-tolerant messaging system. It enables the efficient, real-time processing of large streams of data across multiple applications or systems. Kafka's architecture is built around topics, which are logical channels for publishing and subscribing to data streams. It features horizontal scalability, allowing it to handle enormous volumes of data and supporting high throughput and low latency. Kafka's key components include producers that publish data to topics, consumers that subscribe to topics and process data, and brokers that manage the storage and distribution of data across the cluster. Its durable and fault-tolerant design ensures data integrity and reliability, even in the event of node failures. Kafka's versatility extends beyond messaging to support use cases such as event sourcing, stream processing, and real-time analytics. With its rich ecosystem of connectors and integration capabilities, Kafka seamlessly integrates with various data systems and applications. Backed by a thriving open-source community, Kafka continues to evolve and innovate, making it a foundational tool for building scalable and resilient data pipelines in modern distributed systems.

3) Spring Boot: Spring Boot is a framework for building Java-based applications with minimal configuration and boilerplate code. It simplifies the development process by providing a streamlined setup for creating standalone, production-ready applications. Spring Boot leverages the Spring framework's core features, such as dependency injection and aspect-oriented programming, to facilitate rapid development while maintaining flexibility and extensibility. With its embedded server support, Spring Boot eliminates the need for external application servers, enabling developers to package and deploy applications as executable JAR files. It promotes convention over configuration, offering sensible defaults and auto-configuration capabilities to minimise setup overhead. Spring Boot also provides a wide range of starter dependencies and libraries, allowing developers to quickly integrate popular technologies and frameworks into their applications. Additionally, it includes robust support for microservices architecture, enabling developers to build and deploy scalable, cloud-native applications with ease. Backed by a vibrant community and extensive documentation, Spring Boot continues to be a preferred choice for Java developers seeking simplicity, productivity, and maintainability in their application development endeavours.

4) Containerisation with Docker and Kubernetes: Docker is a leading platform for containerisation, allowing developers to package applications and their dependencies into lightweight, portable containers. These containers encapsulate everything needed to run an application, including code, runtime, system tools, and libraries, ensuring consistency across different environments. Docker simplifies the development, deployment, and scaling of applications by abstracting away differences in operating systems and hardware. It enables developers to build, ship, and run applications seamlessly across various infrastructure environments, from local development machines to public clouds. Kubernetes, often abbreviated as K8s, is an open-source container orchestration platform designed to automate the deployment, scaling, and management of containerised applications. Kubernetes provides a highly scalable and resilient infrastructure for deploying containers across clusters of machines, abstracting away the underlying complexity of managing containerised workloads. It offers features such as automated scaling, self-healing, service discovery, and load balancing, making it ideal for running distributed, microservices-based architectures. Together, Docker and Kubernetes form a powerful combination for building, deploying, and managing modern applications. Docker provides the tools for containerising applications, while Kubernetes orchestrates and manages these containers at scale. This combination enables organisations to achieve greater efficiency, agility, and scalability in their software delivery pipelines, ultimately accelerating the pace of innovation and improving the resilience of their applications in production environments.

B. Linear Temporal Logic monitor

Although RVElastic is formalism-agnostic, a monitor component needs to be instantiated to achieve a self-contained instantiation. In our initial prototype, we decided to use a Linear Temporal Logic monitor.

Linear Temporal Logic (LTL), introduced by Pnueli [16], is a formalism used for specifying and verifying temporal properties of systems. It extends classical propositional logic with temporal operators to express properties about the sequence of states in a system. LTL is particularly useful for reasoning about the future behaviour of reactive systems, such as hardware circuits, communication protocols, and software programs, making it a valuable tool in formal verification and model checking.

Presenting the entire syntax and semantics of LTL is beyond the scope of this contribution, primarily due to space constraints. However, in the next section, we provide and explain an example of an LTL property that has been verified in our case study.

To implement the LTL monitor in RVElastic, the Reelay Python library available at https://doganulus.github.io/reelay/ has been used.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 14:42:31 UTC from IEEE Xplore. Restrictions apply.

V. Case Study

To experiment with our framework, we consider a case study within the context of a distributed document management system. Specifically, we focus on verifying the proper functioning of the authorisation system within document software, which regulates document access based on user roles.

This verification is crucial for ensuring the system's correct operation and for ongoing monitoring. The system classifies users into three types: normal users, editor users, and administrators. Additionally, the system defines six types of actions that can be performed on individual documents: View, Lock, Unlock, Edit, Delete, and Print.

The roles required for each action are explained in Table I.

Action	Description	Role
View	Visualise the document	All
Lock	Lock the document for edit	Editor and Admin
Unlock	Remove the lock for the document	Editor and Admin
Edit	Edit the document	Editor and Admin
Delete	Delete the document	Admin
Print	Print the document	All

TABLE I: Mapping role actions on the system.

The most intriguing aspect concerns the document editing process. To edit a document, it must initially be locked, then edited, and subsequently unlocked. If a user attempts to edit a document without first locking it, or if it is already locked by another user, the system must display an error. Similarly, if a user attempts to lock an object already locked by another user or remove the lock from an object locked by another user, the system must also generate an error. Additionally, the system must reject actions if the user lacks the appropriate role to perform a specific operation.

This requirement can be expressed with the following LTL formula:

$$\mathbf{G}(edit \vee delete \;\Rightarrow\; \neg unlock\ \mathbf{S}\ lock)$$

This formula states that it is always true ($\mathbf{G}$) that if either the $edit$ or $delete$ actions are performed, then a $lock$ action was previously performed and no $unlock$ action has been observed since ($\mathbf{S}$). Note that, other LTL formulas have been verified in the case study as well, but are not reported due to space constraints. However, they can be found along with the previous property in the GitHub repository.

Now that we have identified the property we want to verify, we can focus on the type of events propagated from the system to the monitor. Specifically, the system generates a log of the actions executed by publishing JSON messages in the format shown in Listing 1 on a designated topic.

Listing 1: JSON event's template.

```
{
  "timestamp": "YYYY-MM-DDThh:mm:ssTZD",
  "hostname": "string",
  "username": "string",
  "source": "string",
  "session": "string",
  "service": "string",
  "action": {
    "alias": "string",
    "name" : "string"
  },
  "result": "0",
  "object": {
    "name": "string"
  }
}
```

The message is captured by the worker and sent to the monitor for analysis. The outcome of the analysis, whether true or false, is incorporated into a JSON attribute **verdict**, enhancing the input message, as reported in Listing 2.

Listing 2: JSON event's template with monitor verdict.

```
{
  "timestamp": "YYYY-MM-DDThh:mm:ssTZD",
  "hostname": "string",
  "username": "string",
  "source": "string",
  "session": "string",
  "service": "string",
  "action": {
    "alias": "string",
    "name" : "string"
  },
  "result": "0",
  "object": {
    "name": "string"
  },
  "verdict": true|false
}
```

Following this, the system stores the data in an OpenSearch index.

One of the critical aspects to analyse in this scenario is whether the system has incorrectly permitted actions to occur. Within the OpenSearch Dashboard, it is feasible to create a counter that tallies the occurrences of messages with a verdict of true. Furthermore, it is possible to create a detailed table showcasing the number of actions with a true verdict categorised by action type. This dashboard is illustrated in Figure 3.

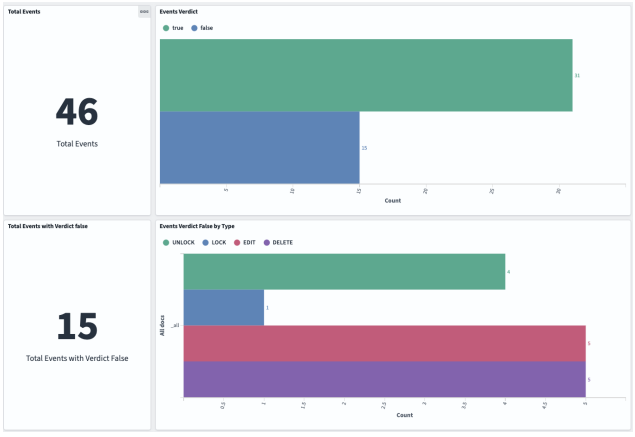


Fig. 3: The OpenSearch Dashboard.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 14:42:31 UTC from IEEE Xplore. Restrictions apply.

A. Experimental evaluation

In addition to testing our approach on the previously presented case study, we decided to conduct further experimental evaluations of its performance. Specifically, in RV, it is common to analyse the overhead introduced by the monitors, *i.e.*, how the system is negatively affected by having monitors running within it. Different aspects can be considered when discussing overhead, but in our case, we are interested in the message passing delay. Therefore, we aim to analyse whether the presence of monitors causes delays in the transmission of messages in the system. Since the system under analysis is distributed and based on the notion of message passing, we chose to focus on this specific aspect.



Fig. 4: Overhead evaluation.

Figure 4 shows the overhead observed in the case study. On the x-axis, we have the actions performed (e.g., edit, view, delete, and so on), while on the y-axis, we have the time required for each action to be completed. This represents the time between sending the message requiring the action to be performed and receiving the message reporting the result of the operation. We analysed the system both in the presence and absence of a monitor. As can be observed, the resulting plot shows minimal interference of the monitors in the system; indeed, the two plots overlap almost everywhere (excluding noise).

The reason for the negligible overhead lies in the monitored system being asynchronous to the running system. In fact, the writer and worker components can scale horizontally to increase the speed of event analysis. Thus, as the system grows in size, the writer and worker components also scale accordingly.

VI. Conclusions and Future Work

This paper presents RVElastic, a Runtime Verification framework tailored specifically for monitoring microservice systems. By leveraging Apache Kafka for system instrumentation and OpenSearch for analysis and visualization of verification results, RVElastic offers a scalable and flexible solution for ensuring the safety and reliability of microservice-based systems.

Through experimentation on a simple case study, we have demonstrated the effectiveness of RVElastic in detecting anomalies, violations, and potential safety hazards in real-time. Our evaluation of the overhead introduced by the addition of monitors in the microservice system provides insights into the practical feasibility and performance implications of our approach.

Moving forward, further research is warranted to explore the scalability and applicability of RVElastic to larger and more complex microservice architectures. Additionally, investigating techniques for integrating RVElastic into existing development and deployment workflows would be valuable for enabling its adoption in industry settings.

References

[1] S. Newman, *Building microservices - designing fine-grained systems, 1st Edition*. O'Reilly, 2015.

[2] N. C. Mendonça, P. Jamshidi, D. Garlan, and C. Pahl, "Developing self-adaptive microservice systems: Challenges and directions," *IEEE Softw.*, vol. 38, no. 2, pp. 70–79, 2021.

[3] A. Maurya and D. Kumar, "Reliability of safety-critical systems: A state-of-the-art review," *Qual. Reliab. Eng. Int.*, vol. 36, no. 7, pp. 2547–2568, 2020.

[4] E. Bartocci, Y. Falcone, A. Francalanza, and G. Reger, "Introduction to runtime verification," in *Lectures on Runtime Verification - Introductory and Advanced Topics* (E. Bartocci and Y. Falcone, eds.), vol. 10457 of *Lecture Notes in Computer Science*, pp. 1–33, Springer, 2018.

[5] N. Yaseen, B. Arzani, R. Beckett, S. Ciraci, and V. Liu, "Aragog: Scalable runtime verification of shardable networked systems," in *14th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2020, Virtual Event, November 4-6, 2020*, pp. 701–718, USENIX Association, 2020.

[6] N. Shafiei, K. Havelund, and P. C. Mehlitz, "Actor-based runtime verification with MESA," in *Runtime Verification - 20th International Conference, RV 2020, Los Angeles, CA, USA, October 6-9, 2020, Proceedings* (J. Deshmukh and D. Nickovic, eds.), vol. 12399 of *Lecture Notes in Computer Science*, pp. 221–240, Springer, 2020.

[7] S. Kauffman, K. Havelund, and R. Joshi, "nfer - A notation and system for inferring event stream abstractions," in *Runtime Verification - 16th International Conference, RV 2016, Madrid, Spain, September 23-30, 2016, Proceedings* (Y. Falcone and C. Sánchez, eds.), vol. 10012 of *Lecture Notes in Computer Science*, pp. 235–250, Springer, 2016.

[8] D. A. Basin, M. Gras, S. Krstic, and J. Schneider, "Scalable online monitoring of distributed systems," in *Runtime Verification - 20th International Conference, RV 2020, Los Angeles, CA, USA, October 6-9, 2020, Proceedings* (J. Deshmukh and D. Nickovic, eds.), vol. 12399 of *Lecture Notes in Computer Science*, pp. 197–220, Springer, 2020.

[9] D. Dams, K. Havelund, and S. Kauffman, "Runtime verification as documentation," in *Leveraging Applications of Formal Methods, Verification and Validation. Software Engineering - 11th International Symposium, ISoLA 2022, Rhodes, Greece, October 22-30, 2022, Proceedings, Part II* (T. Margaria and B. Steffen, eds.), vol. 13702 of *Lecture Notes in Computer Science*, pp. 157–173, Springer, 2022.

[10] I. Cassar, A. Francalanza, L. Aceto, and A. Ingólfsdóttir, "A survey of runtime monitoring instrumentation techniques," in *Proceedings Second International Workshop on Pre- and Post-Deployment Verification Techniques, PrePost@iFM 2017, Torino, Italy, 19 September 2017* (A. Francalanza and G. J. Pace, eds.), vol. 254 of *EPTCS*, pp. 15–28, 2017.

[11] T. S. Team, "Spring boot," 2024. Accessed on March 26, 2024.

[12] The Apache Software Foundation, "Apache kafka," 2024. Accessed on March 26, 2024.

[13] OpenSearch, "Opensearch," 2024. Accessed on March 26, 2024.

[14] Docker Inc., "Docker," 2024. Accessed on March 26, 2024.

[15] Kubernetes, "Kubernetes," 2024. Accessed on March 26, 2024.

[16] A. Pnueli, "The temporal logic of programs," in *18th Annual Symposium on Foundations of Computer Science, Providence, Rhode Island, USA, 31 October - 1 November 1977*, pp. 46–57, IEEE Computer Society, 1977.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 14:42:31 UTC from IEEE Xplore. Restrictions apply.